

NUMPY

```
In [4]: import numpy as np
```

- Array : Collection of same type of data (Homogeneous data).

1D Array :

```
In [7]: a = np.array([10, 20, 30, 50, 60,0])
```

Attributes of arrays

- dtype
- ndim
- size
- shape

```
In [9]: a.dtype
```

```
Out[9]: dtype('int64')
```

```
In [10]: a.shape
```

```
Out[10]: (6,)
```

```
In [11]: a.ndim
```

```
Out[11]: 1
```

```
In [12]: a.size
```

```
Out[12]: 6
```

```
In [13]: a[0]
```

```
Out[13]: np.int64(10)
```

```
In [14]: a[2]
```

```
Out[14]: np.int64(30)
```

Multi dimensional array :

```
In [16]: a_2d= np.array([[10,20,30],[23,55,15]])
```

```
In [17]: a_2d
```

```
Out[17]: array([[10, 20, 30],  
               [23, 55, 15]])
```

```
In [18]: a_2d.dtype
```

```
Out[18]: dtype('int64')
```

```
In [19]: a_2d.shape
```

```
Out[19]: (2, 3)
```

```
In [20]: a_2d.ndim
```

```
Out[20]: 2
```

```
In [21]: a_2d.size
```

```
Out[21]: 6
```

```
In [22]: a_2d[0]
```

```
Out[22]: array([10, 20, 30])
```

Another type of multi dimensional arrays

```
In [24]: another_type_array = np.array([[[1,2,3,4]]])
```

```
In [25]: another_type_array
```

```
Out[25]: array([[[1, 2, 3, 4]]])
```

```
In [26]: another_type_array.shape
```

```
Out[26]: (1, 1, 4)
```

```
In [27]: another_type_array.ndim
```

```
Out[27]: 3
```

First element in first array :

```
In [29]: a_2d
```

```
Out[29]: array([[10, 20, 30],
               [23, 55, 15]])
```

```
In [30]: a_2d[0][0]
```

```
Out[30]: np.int64(10)
```

Second element in second array

```
In [32]: a_2d
```

```
Out[32]: array([[10, 20, 30],
               [23, 55, 15]])
```

```
In [33]: a_2d[1][1]
```

```
Out[33]: np.int64(55)
```

Lets change its data type into float

```
In [35]: a_2d
```

```
Out[35]: array([[10, 20, 30],
               [23, 55, 15]])
```

```
In [36]: a_2d=np.array([[10,20,30,], [23,55,15]], dtype = float)
```

```
In [37]: a_2d
```

```
Out[37]: array([[10., 20., 30.],
               [23., 55., 15.]])
```

Lets create a bigger array

```
In [39]: a_bigger=np.array([
[
[10,20,30,40],[8,8,2,1],[1,1,1,2]
],
[
[9,9,2,39],[1,2,3,3],[0,0,3,2]
],
[
[12,33,22,1],[22,1,22,2],[0,2,3,1]
]
],dtype=float)
```

```
In [40]: a_bigger
```

```
Out[40]: array([[[10., 20., 30., 40.],
                 [ 8.,  8.,  2.,  1.],
                 [ 1.,  1.,  1.,  2.]],

                [[ 9.,  9.,  2., 39.],
                 [ 1.,  2.,  3.,  3.],
                 [ 0.,  0.,  3.,  2.]],

                [[12., 33., 22.,  1.],
                 [22.,  1., 22.,  2.],
                 [ 0.,  2.,  3.,  1.]])
```

- shape in 3-D array is calculated in (No.of Arrays, No.of Rows, No.of columns) format.

```
In [42]: a_bigger.shape
```

```
Out[42]: (3, 3, 4)
```

```
In [43]: a_bigger.ndim
```

```
Out[43]: 3
```

```
In [44]: a_bigger.size
```

```
Out[44]: 36
```

```
In [45]: a_bigger[2][2][3]
```

```
Out[45]: np.float64(1.0)
```

```
In [46]: a_bigger[2][2]
```

```
Out[46]: array([0., 2., 3., 1.])
```

Filling arrays

```
In [48]: np.full
```

```
Out[48]: <function numpy.full(shape, fill_value, dtype=None, order='C', *, device=None, like=None)>
```

- By using the full function for example, we fill an array of a certain shape with the same number. In this case we create a 3x5x4 matrix, which is filled with sevens.

```
In [50]: a_full = np.full(( 3 , 5, 4 ), 7)
```

```
In [51]: a_full
```

```
Out[51]: array([[[7, 7, 7, 7],
                 [7, 7, 7, 7],
                 [7, 7, 7, 7],
                 [7, 7, 7, 7],
                 [7, 7, 7, 7]],

                [[7, 7, 7, 7],
                 [7, 7, 7, 7],
                 [7, 7, 7, 7],
                 [7, 7, 7, 7],
                 [7, 7, 7, 7]],

                [[7, 7, 7, 7],
                 [7, 7, 7, 7],
                 [7, 7, 7, 7],
                 [7, 7, 7, 7],
                 [7, 7, 7, 7]]])
```

```
In [52]: a_full.ndim
```

```
Out[52]: 3
```

ZEROS AND ONES :

- For the cases that we want arrays full of zeros or ones, we even have specific functions.

```
In [55]: a_zeros = np.zeros((3,3))
a_zeros
```

```
Out[55]: array([[0., 0., 0.],
                [0., 0., 0.],
                [0., 0., 0.]])
```

```
In [56]: b_ones = np.ones((2,3,4))
b_ones
```

```
Out[56]: array([[[1., 1., 1., 1.],
                 [1., 1., 1., 1.],
                 [1., 1., 1., 1.]],

                [[1., 1., 1., 1.],
                 [1., 1., 1., 1.],
                 [1., 1., 1., 1.]])
```

```
In [57]: b_ones.ndim
```

```
Out[57]: 3
```

EMPTY AND RANDOM

- Other options would be to create an empty array or one that is filled with random numbers. For this, we use the respective functions once again.

```
In [60]: e = np.empty((4,4))  
e
```

```
Out[60]: array([[6.23042070e-307, 4.67296746e-307, 1.69121096e-306,  
                6.23058707e-307],  
               [2.22522597e-306, 1.33511969e-306, 1.37962320e-306,  
                9.34604358e-307],  
               [9.79101082e-307, 1.78020576e-306, 1.69119873e-306,  
                2.22522868e-306],  
               [1.24611809e-306, 8.06632139e-308, 1.60221208e-306,  
                2.29178686e-312]])
```

```
In [61]: b_random = np.random.random((2,2))  
b_random
```

```
Out[61]: array([[0.2025059 , 0.83738864],  
               [0.71153464, 0.97572501]])
```

```
In [62]: b_random.ndim
```

```
Out[62]: 2
```

```
In [63]: b_random.size
```

```
Out[63]: 4
```

RANGES

- Instead of just filling arrays with the same values, we can fill or create sequences of values by specifying the boundaries. For this, we can use two different functions, namely `arange` and `linspace`.

Arange :

- Arguments are: start-stop -step (increment)

```
In [68]: a_arange = np.arange(10,50, 5)
```

```
In [69]: a_arange
```

```
Out[69]: array([10, 15, 20, 25, 30, 35, 40, 45])
```

linspace :

- By using linspace we also create a list from a minimum value to a maximum value. But instead of specifying the step-size, we specify the amount of values that we want to have in our list. They will all be spread evenly and have the same distance to their neighbors.
- We can specify the starting point and ending point, then the linspace() method will return the random values between the range we inputed.
- start-stop-number of values

```
In [73]: b_linspace = np.linspace(0, 50, 2)
b_linspace
```

```
Out[73]: array([ 0., 50.])
```

```
In [74]: b_linspace.ndim
```

```
Out[74]: 1
```

```
In [75]: b_linspace.size
```

```
Out[75]: 2
```

```
In [76]: b_linspace1 = np.linspace(0, 50, 30)
b_linspace1
```

```
Out[76]: array([ 0.          ,  1.72413793,  3.44827586,  5.17241379,  6.89655172,
                8.62068966, 10.34482759, 12.06896552, 13.79310345, 15.51724138,
                17.24137931, 18.96551724, 20.68965517, 22.4137931 , 24.13793103,
                25.86206897, 27.5862069 , 29.31034483, 31.03448276, 32.75862069,
                34.48275862, 36.20689655, 37.93103448, 39.65517241, 41.37931034,
                43.10344828, 44.82758621, 46.55172414, 48.27586207, 50.          ])
```

```
In [77]: b_linspace1.size
```

```
Out[77]: 30
```

```
In [78]: b_linspace2 = np.linspace(0, 50, 25, dtype=int)
b_linspace2
```

```
Out[78]: array([ 0,  2,  4,  6,  8, 10, 12, 14, 16, 18, 20, 22, 25, 27, 29, 31, 33,
                35, 37, 39, 41, 43, 45, 47, 50])
```

Identity Matrix :

```
In [80]: np.identity (5)
```

```
Out[80]: array([[1., 0., 0., 0., 0.],
               [0., 1., 0., 0., 0.],
               [0., 0., 1., 0., 0.],
               [0., 0., 0., 1., 0.],
               [0., 0., 0., 0., 1.]])
```

OR

```
In [82]: np.eye(3)
```

```
Out[82]: array([[1., 0., 0.],
               [0., 1., 0.],
               [0., 0., 1.]])
```

Math functions :

- Takes e to the power of each values

```
In [187... a
```

```
Out[187... array([10, 20, 30, 50, 60, 0])
```

```
In [85]: np.exp(a)
```

```
Out[85]: array([2.20264658e+04, 4.85165195e+08, 1.06864746e+13, 5.18470553e+21,
               1.14200739e+26, 1.00000000e+00])
```

- Returns the sine of each value

```
In [87]: np.sin(a)
```

```
Out[87]: array([-0.54402111,  0.91294525, -0.98803162, -0.26237485, -0.30481062,
               0.          ])
```

- Returns the cosine of each value

```
In [89]: np.cos(a)
```

```
Out[89]: array([-0.83907153,  0.40808206,  0.15425145,  0.96496603, -0.95241298,
               1.          ])
```

- Returns the tangent of each value

```
In [91]: np.tan(a)
```



```
Out[91]: array([ 0.64836083,  2.23716094, -6.4053312 , -0.27190061,  0.32004039,
                0.          ])
```

- Returns the logarithm of each value

```
In [93]: np.log(a)
```

```
C:\Users\hp\AppData\Local\Temp\ipykernel_9804\176755284.py:1: RuntimeWarning: divide
by zero encountered in log
  np.log(a)
```

```
Out[93]: array([2.30258509, 2.99573227, 3.40119738, 3.91202301, 4.09434456,
                -inf])
```

- Returns the square root of each value

```
In [191... np.sqrt(a)
```

```
Out[191... array([3.16227766, 4.47213595, 5.47722558, 7.07106781, 7.74596669,
                0.          ])
```

AGGREGATE FUNCTIONS :

- Returns the sum of all values in the array

```
In [98]: a.sum()
```

```
Out[98]: np.int64(170)
```

- Returns the lowest value of the array

```
In [100... a.min()
```

```
Out[100... np.int64(0)
```

- Returns the highest value of the array

```
In [102... a.max()
```

```
Out[102... np.int64(60)
```

- Returns the arithmetic mean of all values in the array

```
In [104... a.mean()
```

```
Out[104... np.float64(28.333333333333332)
```

- Returns the median value of the array

```
In [106...] np.median(a)
```

```
Out[106...] np.float64(25.0)
```

- Returns the standard deviation of the values in the array

```
In [108...] np.std(a)
```

```
Out[108...] np.float64(21.147629234082533)
```

Create some arrays and apply all the functions :

```
In [110...] np.median (b_linspace2)
```

```
Out[110...] np.float64(25.0)
```

```
In [111...] np.sum (b_linspace2)
```

```
Out[111...] np.int64(614)
```

```
In [112...] np.sqrt(b_linspace2)
```

```
Out[112...] array([0.          , 1.41421356, 2.          , 2.44948974, 2.82842712,
 3.16227766, 3.46410162, 3.74165739, 4.          , 4.24264069,
 4.47213595, 4.69041576, 5.          , 5.19615242, 5.38516481,
 5.56776436, 5.74456265, 5.91607978, 6.08276253, 6.244998   ,
 6.40312424, 6.55743852, 6.70820393, 6.8556546 , 7.07106781])
```

```
In [113...] np.std(b_linspace2)
```

```
Out[113...] np.float64(14.924020905908701)
```

```
In [114...] np.cos(b_linspace2)
```

```
Out[114...] array([ 1.          , -0.41614684, -0.65364362,  0.96017029, -0.14550003,
 -0.83907153,  0.84385396,  0.13673722, -0.95765948,  0.66031671,
  0.40808206, -0.99996083,  0.99120281, -0.29213881, -0.74805753,
  0.91474236, -0.01327675, -0.90369221,  0.76541405,  0.26664293,
 -0.98733928,  0.55511133 ,  0.52532199, -0.99233547,  0.96496603])
```

```
In [ ]:
```